

U23CC352 – DISCRETE TIME SIGNAL PROCESSING LABORATORY

EXPERIMENTS USING MATLAB / EQUIVALENT SOFTWARE PACKAGE

1. Compute the Linear and Circular convolution of the any two sequences.
2. Compare the computational complexity of DFT and FFT algorithm.
3. Design and demonstrate the filtering operation of Butterworth IIR filters (LPF/HPF/BPF/BSF).
4. Design and demonstrate the filtering operation of Chebyshev IIR filters (LPF/HPF/BPF/BSF).
5. Design and demonstrate the filtering operation of FIR filters (LPF/HPF/BPF/BSF) using windowing techniques.

EXPERIMENTS USING DIGITAL SIGNAL PROCESSOR

6. Study of architecture of Digital Signal Processor (TMS320C67XX).
7. Perform MAC operation using various addressing modes.
8. Generate various signals and random noise.
9. Implementation of Butter worth and Chebyshev IIR Filters for Low pass, High pass, Band pass and Band stop filtering.
10. Implementation of FIR Filter for Low pass, High pass, Band pass and Band stop Filtering DSP.

Date :

AIM:

To write a program to perform the linear and circular convolution of two sequences.

APPARATUS REQUIRED:

Computer with MATLAB / OCTAVE

ALGORITHM:

LINEAR CONVOLUTION:

1. Get the number of samples.
2. Generate the output sequence of the given two input signals using 'conv' command.
3. Plot the graph.

CIRCULAR CONVOLUTION

1. Get the number of samples.
2. Generate the sequence for the given two input signals using 'zeros' and 'ones' matrix command.
3. Generate the convoluted output sequence of the given two input signal using 'conv' command.
4. Plot the graph.

PROGRAM

LINEAR CONVOLUTION

```
clc;
x=input('enter 1st seq');
h=input('enter 2nd seq');
y=conv(x,h);
figure;
subplot(3,1,1);
stem(x);
ylabel('amplitude');
xlabel('a n--->');
subplot(3,1,2);
stem(h);
ylabel('amplitude');
xlabel('b n--->');
subplot(3,1,3);
stem(y);
ylabel('amplitude');
xlabel('c n--->');
disp('the result is');y
```

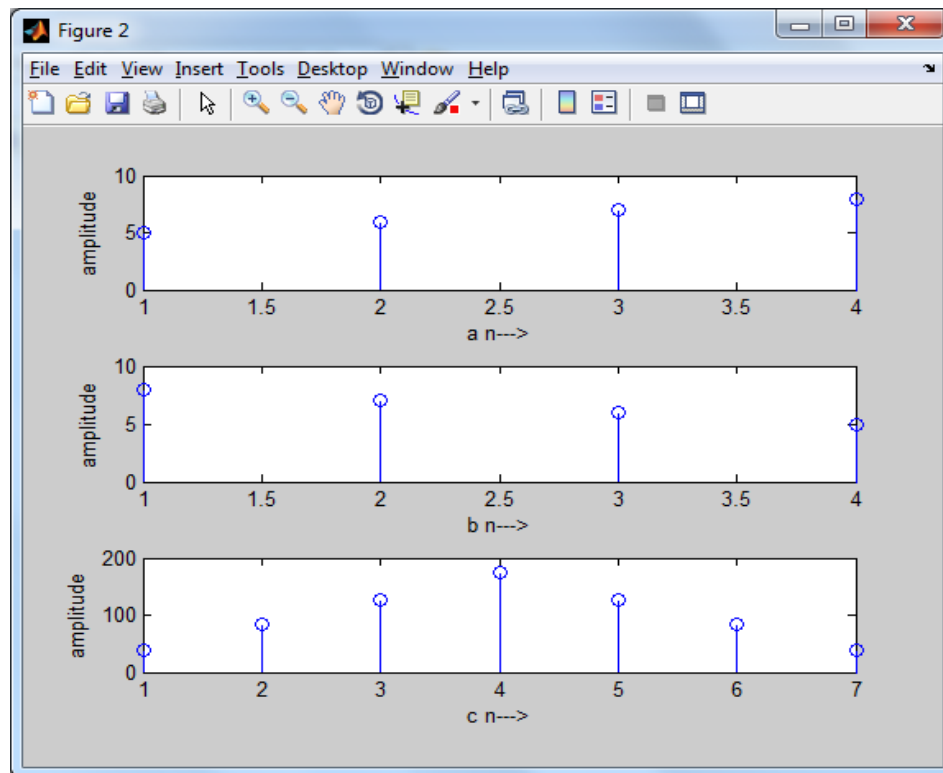
CIRCULAR CONVOLUTION

```
clc;
clearall;
x=[1 1 2 1 1];
h=[1 1 2 1];
Nx=length(x);
Nh=length(h);
N=max(Nx,Nh);
y=cconv(x,h,N);
n=0:1:Nx-1;
subplot(2,2,1);
stem(n,x);
xlabel('n');
ylabel('x(n)');
title('input sequence');
n=0:1:Nh-1;
subplot(2,2,2);
stem(n,h);
xlabel('n');
ylabel('h(n)');
title('impulse sequence');
n=0:1:N-1;
subplot(2,2,3);
stem(n,y);
title('circular convolution');
```

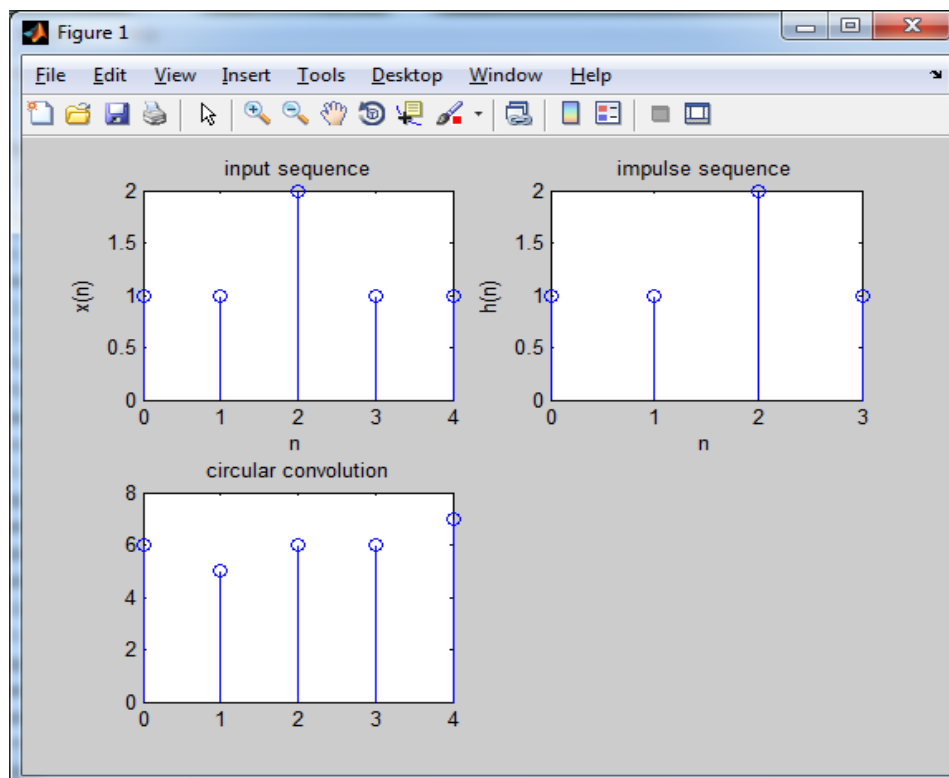
OUTPUT

LINEAR CONVOLUTION

enter 1st seq [5 6 7 8]
 enter 2nd seq [8 7 6 5]
 the result is
 $y = 40 \quad 83 \quad 128 \quad 174 \quad 128 \quad 83 \quad 40$



CIRCULAR CONVOLUTION



Ex.No : 02

CALCULATION OF FFT

Date :

AIM:

To write a program for the calculation of FFT.

APPARATUS REQUIRED:

Computer with MATLAB / OCTAVE

ALGORITHM:

1. Get the input sequence and its length.
2. Calculate the Fast Fourier Transform and Inverse Fast Fourier Transform of the given sequence.

PROGRAM

```
clc;
clear all;
% Fast fourier transform
disp('Fast fourier transform');
l=input('enter the length ');
x=input('enter the input sequence');
y=fft(x,l);
disp('resultant FFT sequence');y
subplot(2,2,1);
stem(x);
title('input');xlabel('n-?'); ylabel('x(n)');
subplot(2,2,2);
stem(y);
title('FFT sequence'); xlabel('frequency'); ylabel('X(k)');

%Inverse Fast fourier transform
disp('inverse fast fourier transform');
x1=input('enter the FFT sequence');
y1=ifft(x1);
disp('resultant IFFT sequence');y1
subplot(2,2,3);
stem(x1);
title('FFT sequence'); xlabel('frequency'); ylabel('X(k)');
subplot(2,2,4);
stem(y1);
title('IFFT sequence'); xlabel('n--?'); ylabel('x(n)');
figure(2);
freqz(y);
title('fft sequence');
figure(3);
freqz(y1);
title('ifft sequence');
```

OUTPUT

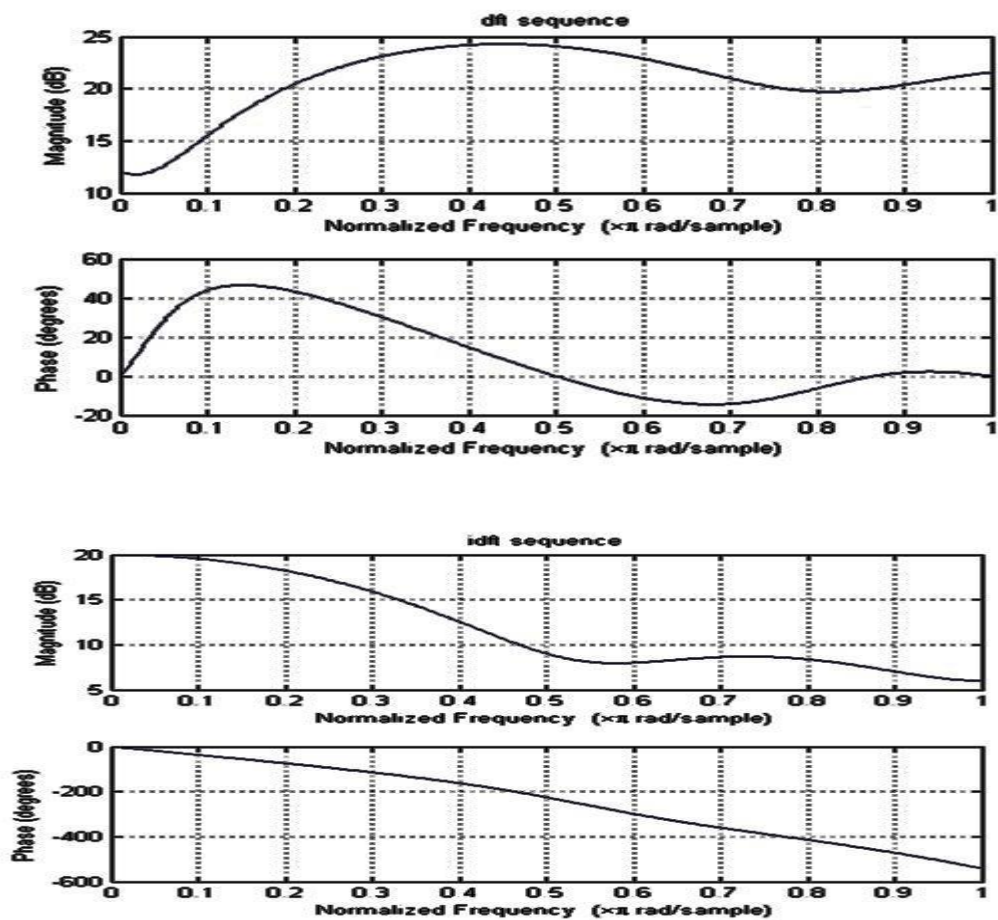
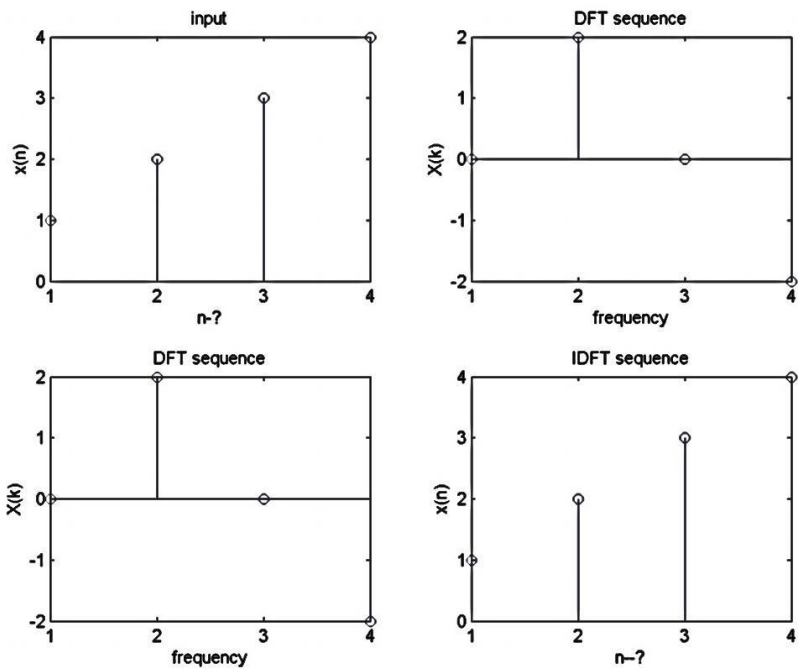
Length of the sequence = 4

Input Sequence for FFT= [1, 2, 3, 4]

FFT Output Sequence = $[10, -2 + 2i, -2, 2 - 2i]$

Input Sequence for IFFT = $[10, -2 + 2i, -2, 2 - 2i]$

IFFT Output Sequence = $[1, 2, 3, 4]$



Ex.No: 03

DESIGN OF BUTTERWORTH IIR FILTER

Date:

AIM:

To write a program to design a Butterworth IIR filter.

APPARATUS REQUIRED:

Computer with MATLAB / OCTAVE

ALGORITHM:

1. Get the pass band and stop band normalized frequency.
2. Get the pass band and stop band attenuation level.
3. Plot the magnitude and phase response graph.

PROGRAM

BUTTERWORTH LOW PASS FILTER

```
clc;
close all;
clear all;
format long
rp=input('enter the passband ripple (rp)=');
rs=input('enter the stopband ripple (rs)=');
wp=input('enter the passbandfreq (wp)=');
ws=input('enter the stopbandfreq (ws)=');
fs=input('enter the sampling freq (fs)=');
w1=2*wp/fs;
w2=2*ws/fs;
[n,wn]=buttord(w1,w2,rp,rs);
[b,a]=butter(n,wn);
[bz,az]=impinvar(b,a,fs);
disp('bz = ');
disp(bz);
disp('az = ');
disp(az);
Hz=tf(bz,az);
w=0:.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
subplot(2,1,1);
plot(om/pi,m);
ylabel('Gain in db -->');
xlabel('(a) Normalised frequency -->');
subplot(2,1,2);
plot(om/pi,an);
xlabel('(b) Normalised frequency -->');
ylabel('Phase in radians -->');
```

BUTTERWORTH BAND PASS FILTER

```
clc;
close all;
clear all;
format long
rp=input('enter the passband ripple (rp)=');
rs=input('enter the stopband ripple (rs)=');
wp=input('enter the passbandfreq (wp)=');
ws=input('enter the stopbandfreq (ws)=');
fs=input('enter the sampling freq (fs)=');
w1=2*wp/fs;
w2=2*ws/fs;

[n]=buttord(w1,w2,rp,rs);
wn=[w1,w2];
[b,a]=butter(n,wn,'bandpass');
[bz,az]=impinvar(b,a,fs);
disp('bz = ');
disp(bz);
disp('az = ');
disp(az);
Hz=tf(bz,az);
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
figure(1);
subplot(2,1,1);
plot(om/pi,m);
ylabel('Gain in deciBel');
xlabel('Normalized frequency in rad/sec');
title('Magnitude Plot (BPF)');
subplot(2,1,2);
plot(om/pi,an);
xlabel('Normalized frequency in rad/sec');
ylabel('Phase Angle in rad');
title('Phase Plot (BPF)');
```


BUTTERWORTH LOW PASS FILTER

Pass band ripple=0.5

Stop band ripple=50

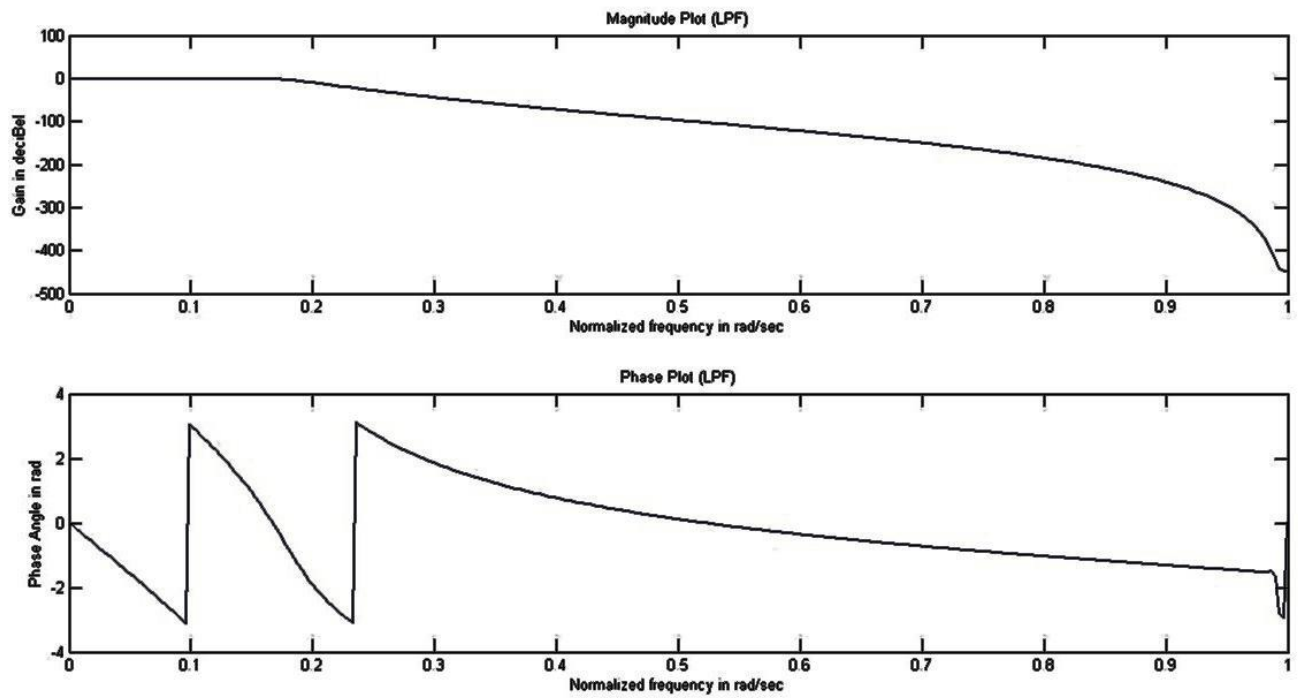
Pass band freq=1200

Stop band freq=2400

Sampling freq=10000

Filter order = 9

Cut-off frequency= 0.179



BUTTERWORTH BAND PASS FILTER

Pass band ripple=0.25

Stop band ripple=20

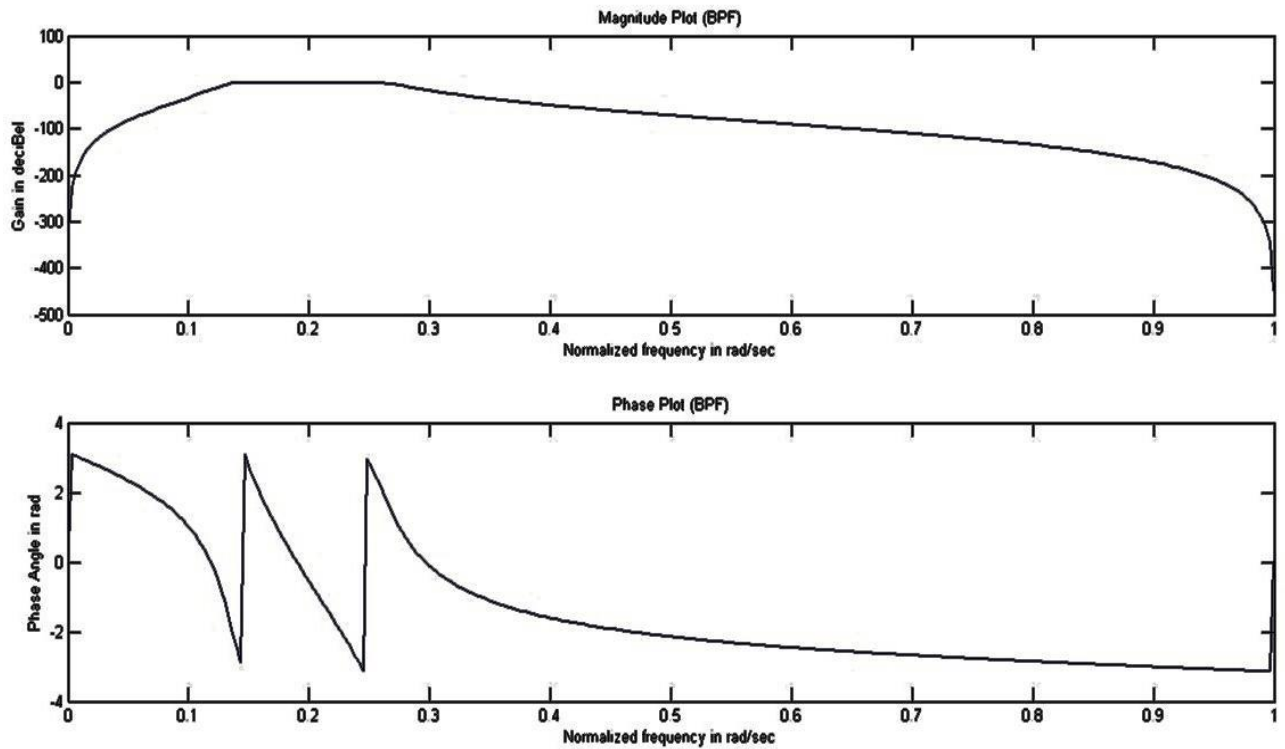
Pass band freq1000

Stop band freq2000

Sampling freq15000

Filter order = 6

Cut-off frequency= [0.133, 0.26]



Ex.No: 04

DESIGN OF CHEBYSHEV IIR FILTER

Date:

AIM:

To write a program to design a Chebyshev IIR filter.

APPARATUS REQUIRED:

Computer with MATLAB / OCTAVE

ALGORITHM:

1. Get the pass band and stop band normalized frequency.
2. Get the pass band and stop band attenuation level.
3. Plot the magnitude and phase response graph.

PROGRAM:

```
clear all;
clc;
close all;
rp=input('Enter the pass band ripple:');
rs=input('Enter the stop band ripple:');
wp=input('Enter the pass band frequency:');
ws=input('Enter the stop band frequency:');
fs=input('Enter the sampling frequency:');
w1=2*wp/fs;
w2=2*ws/fs;
```

%LOW PASS FILTER

```
[n,wn]=cheb1ord(w1,w2,rp,rs);
[b,a]=cheby1(n,rp,wn);
[bz,az]=impinvar(b,a,fs);
disp('bz = ');
disp(bz);
disp('az = ');
disp(az);
Hz=tf(bz,az);
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
disp('Filter order');
disp(n);
disp('Cut-off frequency');
disp(wn);
subplot(2,1,1);
plot(om/pi,m);
ylabel('Gain in db--->');
xlabel('(a)Normalized frequency--->');
```

```

title('Low pass filter');
subplot(2,1,2);
plot(om/pi,an);
xlabel('(b)Normalized frequency--->');
ylabel('Phase in radians--->');

```

%BAND PASS FILTER

```

clear all;
clc;
close all;
rp=input('Enter the pass band ripple:');
rs=input('Enter the stop band ripple:');
wp=input('Enter the pass band frequency:');
ws=input('Enter the stop band frequency:');
fs=input('Enter the sampling frequency:');
w1=2*wp/fs;
w2=2*ws/fs;
[n,wn]=cheb1ord(w1,w2,rp,rs);
[b,a]=cheby1(n,rS,wn,'bandpass');
[bz,az]=impinvar(b,a,fs);
disp('bz = ');
disp(bz);
disp('az = ');
disp(az);
Hz=tf(bz,az);
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
disp('Filter order');
disp(n);
disp('Cut-off frequency');
disp(wn);
subplot(2,1,1);
plot(om/pi,m);
ylabel('Gain in db--->');
xlabel('(a)Normalized frequency--->');
title('Low pass filter');
subplot(2,1,2);
plot(om/pi,an);
xlabel('(b)Normalized frequency--->');

```

CHEBYSHEV LOW PASS FILTER

Passband ripple=0.5

Stopband ripple=50

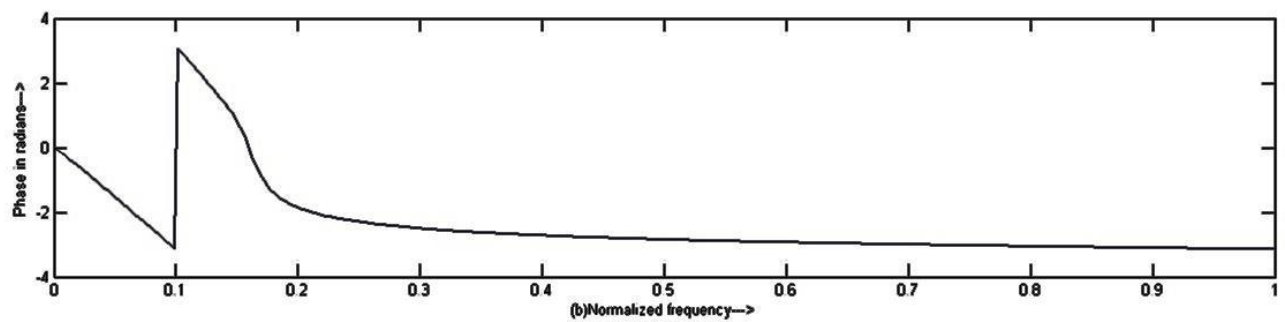
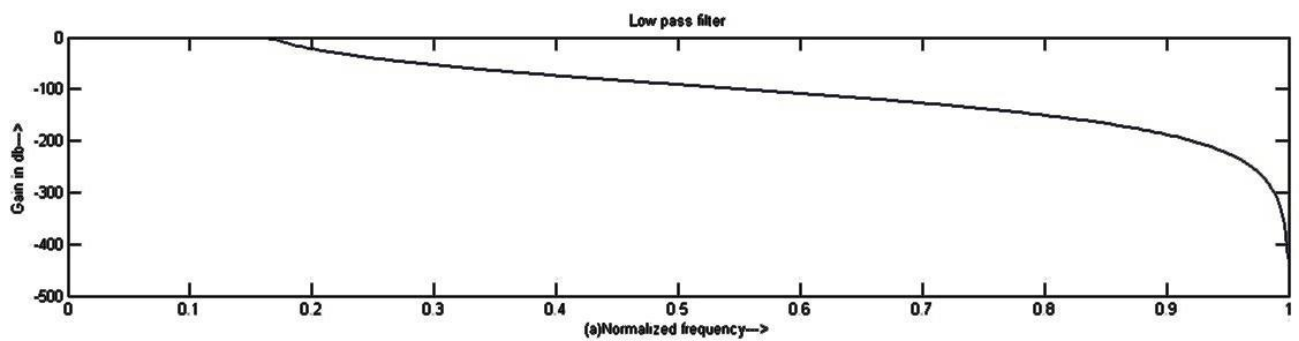
Passband freq=1200

Stop band freq=2400

Sampling freq=15000

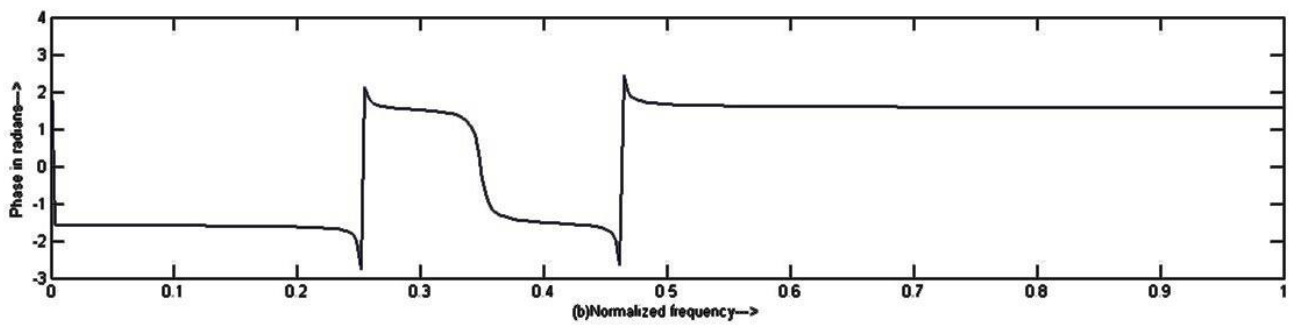
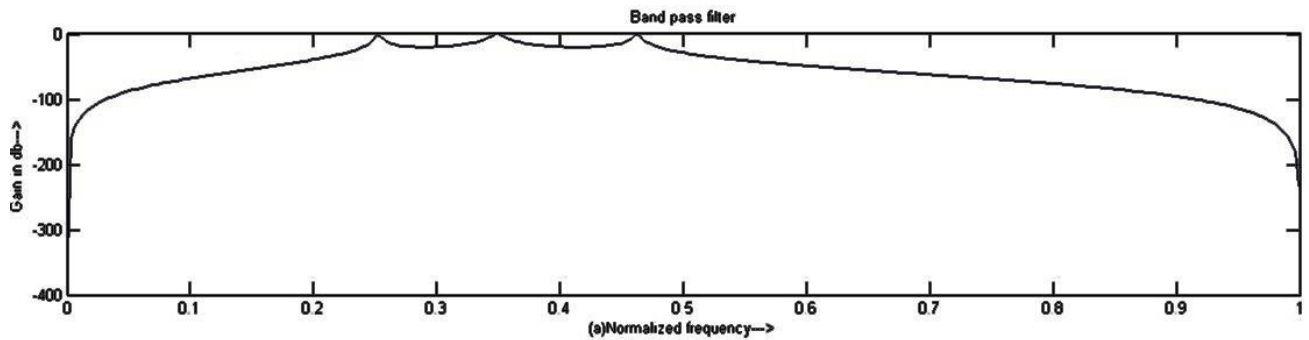
Order of the filter = 6

Cut off frequency= 0.16



CHEBYSHEV BAND PASS FILTER

Passband ripple=0.25
Stopband ripple=20
Passband freq=1200
Stop band freq=2400
Sampling freq=15000
Order of the filter = 3
Cut off frequency= [0.24,0.48]



Ex.No: 05

DESIGN OF FIR FILTER

Date:

AIM:

To write a program to design a FIR (Low pass, High pass, Band pass, Band Stop) filter.

APPARATUS REQUIRED:

Computer with MATLAB / OCTAVE

ALGORITHM:

1. Get the cut off frequency and the order of the filter.
2. Design the Low pass filter using various window techniques.
3. Plot the magnitude response graph.

PROGRAM FOR RECTANGULAR WINDOW

```
clear all;
clc;
disp('design of fir low pass filter using window function');
n=input(' enter number of sample = ');
w=input(' enter the cut of frequency = ');
n=n-1;
%Low Pass Filter
disp('LPF using rectangular window');
lpf=fir1(n,w/pi,rectwin(n+1));
[h,w1]=freqz(lpf,1,256);
m=20*log(abs(h));
subplot(2,2,1);
plot(w1/pi,m);
ylabel('gain in dB');
xlabel('normalized freq');
title('low pass filter');
title('LPF using rectangular window');

% High Pass filter
disp('HPF using rectangular window');
hpf=fir1(n,w/pi,'high',rectwin(n+1));
[h1,w2]=freqz(hpf,1,256);
m1=20*log(abs(h1));
subplot(2,2,2);
plot(w2/pi,m1);
ylabel('gain in dB');
xlabel('normalized freq');
title('High pass filter');
title('HPF using rectangular window');
```

% Band Pass filter

```
wc1=input(' Enter the first cut of frequency = ');
wc2=input(' Enter the second cut of frequency= ');
wn=[wc1 wc2];
disp('BPF using rectangular window');
bpf=fir1(n,wn/pi,'bandpass',rectwin(n+1));
[h2,w3]=freqz(bpf,1,256);
m2=20*log(abs(h2));
subplot(2,2,3);
plot(w3/pi,m2);
ylabel('gain in dB');
xlabel('normalized freq');
title('Band pass filter');
title('BPF using rectangular window');
```

% Band stop filter

```
wc3=input(' Enter the first cut of frequency = ');
wc4=input(' Enter the second cut of frequency = ');
wn1=[wc3 wc4];
disp('BSF using rectangular window');
bsf=fir1(n,wn1/pi,'stop',rectwin(n+1));
[h3,w4]=freqz(bsf,1,256);
m3=20*log(abs(h3));
subplot(2,2,4);
plot(w4/pi,m3);
ylabel('gain in dB');
xlabel('normalized freq');
title('Band Stop filter');title('BSF using rectangular window');
```

PROGRAM FOR HAMMING WINDOW

```
clear all;
clc;
disp('design of fir low pass filter using window function');
n=input('enter number of sample=');
w=input('enter the cut of frequency=');
n=n-1;
```

%Low Pass Filter

```
disp('LPF using Hamming window');
lpf=fir1(n,w/pi,hamming(n+1));
[h,w1]=freqz(lpf,1,256);
m=20*log(abs(h));
subplot(2,2,1);
plot(w1/pi,m);
ylabel('gain in dB');
xlabel('normalized freq');
title('low pass filter');
title('LPF using Hamming window');
```


% High Pass Filter

```
disp('HPF using Hamming window');
hpf=fir1(n,w/pi,'high',hamming(n+1));
[h1,w2]=freqz(hpf,1,256);
m1=20*log(abs(h1));
subplot(2,2,2);
plot(w2/pi,m1);
ylabel('gain in dB');
xlabel('normalized freq');
title('High pass filter');
title('HPF using Hamming window');
```

%Band Pass Filter

```
wc1=input('enter the first cut of frequency');
wc2=input('enter the second cut of frequency');
wn=[wc1 wc2];
disp('Bpf using Hamming window');
bpf=fir1(n,wn/pi,'bandpass',Hamming(n+1));
[h2,w3]=freqz(bpf,1,256);
m2=20*log(abs(h2));
subplot(2,2,3);
plot(w3/pi,m2);
ylabel('gain in dB');
xlabel('normalized freq');
title('Band pass filter');
title('BPF using Hamming window');
```

%Band Stop Filter

```
wc3=input('enter the first cut of frequency');
wc4=input('enter the second cut of frequency');
wn1=[wc3 wc4];
disp('BSF using Hamming window');
bsf=fir1(n,wn1/pi,'stop',Hamming(n+1));
[h3,w4]=freqz(bsf,1,256);
m3=20*log(abs(h3));
subplot(2,2,4);
plot(w4/pi,m3);
ylabel('gain in dB');
xlabel('normalized freq');
title('Stop pass filter');title('BSF using Hamming window');
```

PROGRAM FOR HANNING WINDOW

```
clear all;
clc;
disp('design of fir low pass filter using window function');
n=input('enter number of sample=');
w=input('enter the cut of frequency=');
n=n-1;
```

%Low Pass Filter

```
disp('LPF using Hanning window');
lpf=fir1(n,w/pi,hann(n+1));
[h,w1]=freqz(lpf,1,256);
m=20*log(abs(h));
subplot(2,2,1);
plot(w1/pi,m);
ylabel('gain in dB');
xlabel('normalized freq');
title('low pass filter');
title('LPF using Hanning window');
```

%High Pass Filter

```
disp('HPF using Hanning window');
hpf=fir1(n,w/pi,'high',hann(n+1));
[h1,w2]=freqz(hpf,1,256);
m1=20*log(abs(h1));
subplot(2,2,2);
plot(w2/pi,m1);
ylabel('gain in dB');
xlabel('normalized freq');
title('High pass filter');
title('HPF using Hanning window');
```

%Band Pass Filter

```
wc1=input('enter the first cut of frequency=');
wc2=input('enter the second cut of frequency=');
wn=[wc1 wc2];
disp('BPF using Hanning window');
bpf=fir1(n,wn/pi,'bandpass',hann(n+1));
[h2,w3]=freqz(bpf,1,256);
m2=20*log(abs(h2));
subplot(2,2,3);
plot(w3/pi,m2);
ylabel('gain in dB');
xlabel('normalized freq');
title('Band pass filter');
title('BPF using Hanning window');
```

%Band Stop Filter

```
wc3=input('enter the first cut of frequency=');
wc4=input('enter the second cut of frequency=');
wn1=[wc3 wc4];
disp('BSF using Hanning window');
bsf=fir1(n,wn1/pi,'stop',hann(n+1));
[h3,w4]=freqz(bsf,1,256);
m3=20*log(abs(h3));
subplot(2,2,4);
plot(w4/pi,m3);
ylabel('gain in dB');
xlabel('normalized freq');
title('Stop pass filter');title('BSF using Hanning window');
```

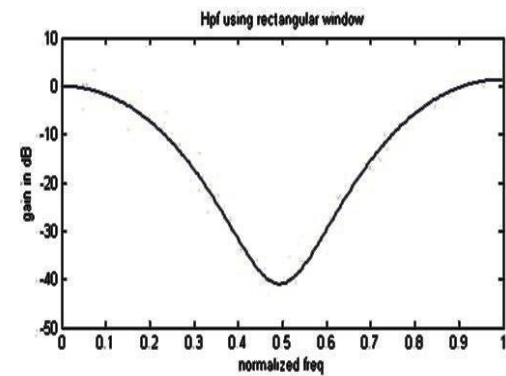
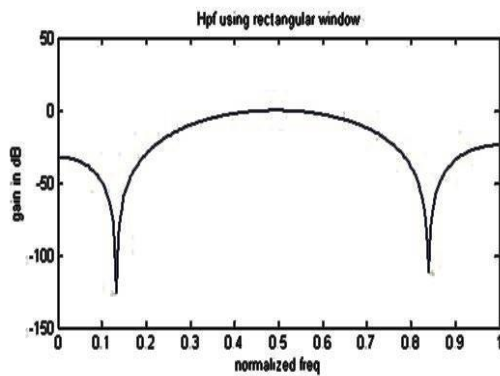
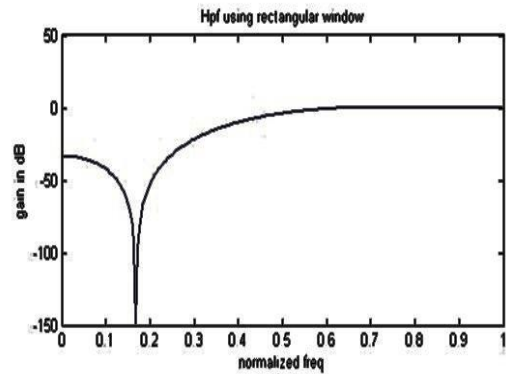
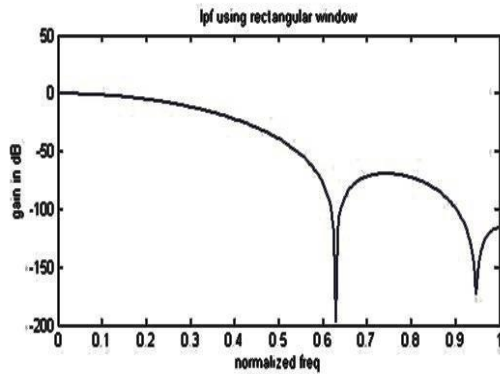
OUTPUT

RECTANGULAR WINDOW:

Number of Samples = 5

Cut off Frequency $W_c = 1.2$ (for LPF and HPF)

Cut off Frequency $W_{c1} = 1$ and $W_{c2} = 2$ (for BPF and BSF)



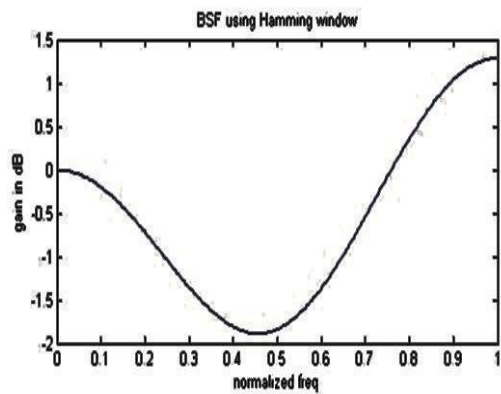
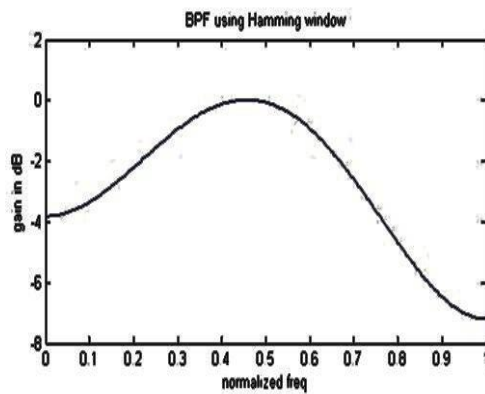
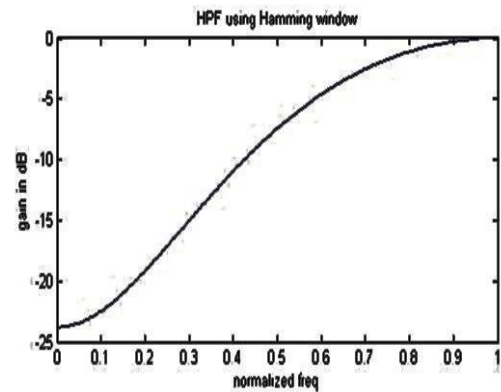
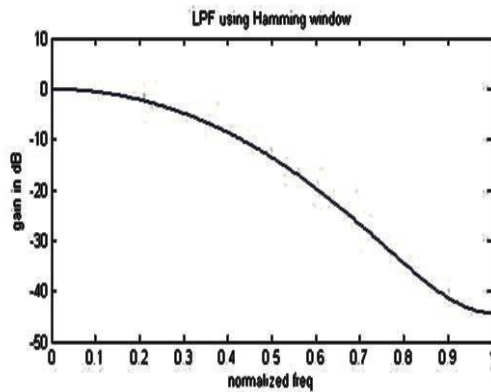
OUTPUT

HAMMING WINDOW:

Number of Samples = 5

Cut off Frequency $W_c = 1.2$ (for LPF and HPF)

Cut off Frequency $W_{c1} = 1$ and $W_{c2} = 2$ (for BPF and BSF)



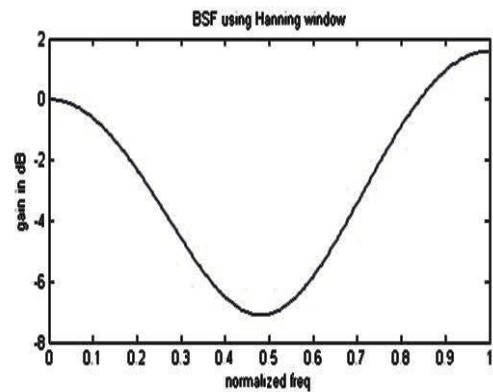
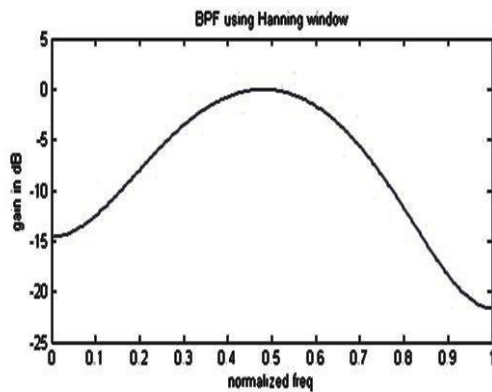
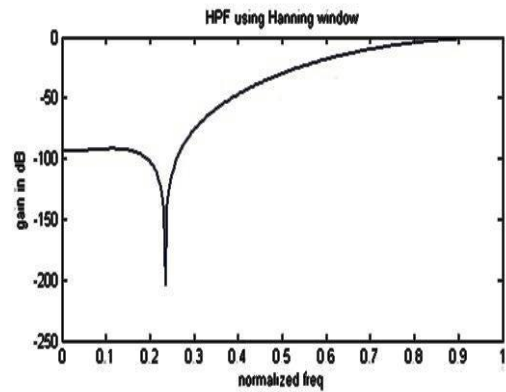
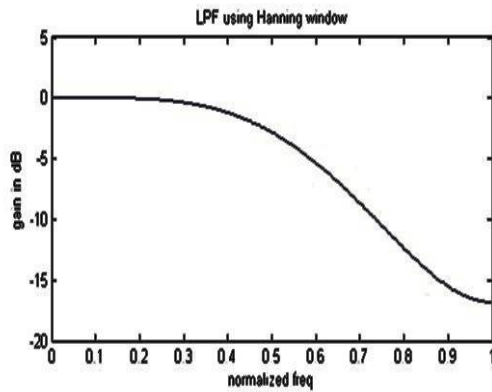
OUTPUT

HANNING WINDOW:

Number of Samples = 5

Cut off Frequency $W_c = 1.2$ (for LPF and HPF)

Cut off Frequency $W_{c1} = 1$ and $W_{c2} = 2$ (for BPF and BSF)



Date:**AIM:**

To study about the architecture of DSP Processor TMS320C6x.

THE DSP CHIP TMS320C6x:

The TMS320C6x devices are the first devices to feature VelociTI0, an advanced very long instruction word (VLIW) architecture developed by Texas Instruments, which allows performance of up to 1600 million instructions per second (MIPS). The first device in the series is the TMS320C6201, a fixed-point digital signal processor (DSP).

With a complete set of development tools, the 'C6x devices offer cost-effective solutions to high-performance DSP programming challenges. The 'C6x development tools include a new C compiler, an assembly optimizer, and a Windows-based debugger. VelociTI combines an advanced VLIW architecture with a high degree of parallelism to produce a device that enables applications such as: Unlimited Internet bandwidth, Universal wireless communications, new telephony features, Remote medical diagnostics, Automated cruise control, Personal home base station, Personalized home security. The 'C6x devices also can be used for improved performance on existing applications, such as: Wireless base stations, Pooled modems and remote access servers, Next-generation xDSL modems and cable modems, Multichannel telephony platforms, including central office switches, PBXs, and voice-messaging systems, Multimedia systems, Digital imaging.

The TMS320C6201 is the first fixed-point processor in the 'C6x generation. Following are key features of the TMS320C6201: VelociTI advanced very long instruction word (VLIW) architecture, Load/store architecture, Instruction packing for reduced code size, 100% conditional instructions for faster execution Intuitive, reduced instruction set computing (RISC)-like instruction set.

Central Processing Unit (CPU)

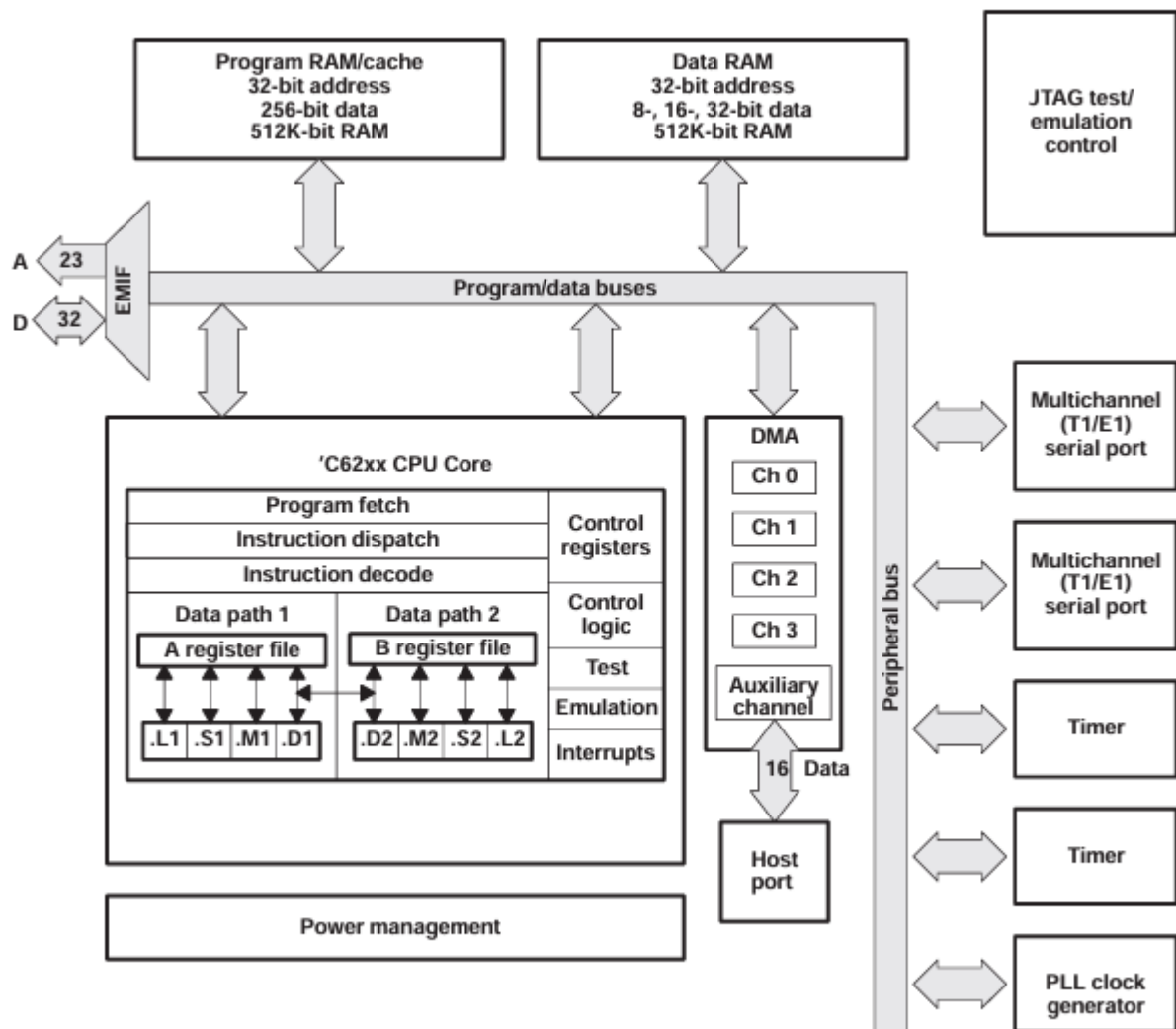
Eight independent functional units (including two 16-bit multipliers with 32-bit results and six arithmetic logic units [ALUs] with 32-/40-bit results), 32 32-bit registers, 1600 million instructions per second (MIPS), Five-ns cycle time, Up to eight 32-bit instructions per cycle, Byte-addressable 8-, 16-, 32-bit data, 32-bit address range, Dual-endian support, Saturation, Normalization, Bit-field instructions (extract, clear, left most bit detection).

The 'C6x central processing unit (CPU) is the central building block of all the TMS320C62xx devices. The CPU contains: Program fetch unit Instruction dispatch unit Instruction decode unit 32 general-purpose, 32-bit registers Two data paths, each with four functional units, including one multiplier and three arithmetic logic units (ALUs) on each data path Control registers Control logic Test, emulation, and interrupt logic The CPU has two data paths where processing occurs. Each data path has four functional units and a register file containing 16 32-bit registers. The functional units execute logic, shifting, multiply, and data address operations. All instructions operate on the registers. The two sets of data-addressing units are exclusively responsible for all data transfers between the register files and the memory.

Addressing Modes:

The addressing mode options on the 'C62xx are either linear or circular, as specified by the addressing-mode register (AMR).

FUNCTIONL BLOCK DIAGRAM:



Interrupts:

The CPU has 14 interrupts. These are the reset interrupt, the nonmaskable interrupt (NMI), and interrupts 4±15. These interrupts correspond to the RESET, NMI, and INT4±INT15 signals on the CPU boundary. In some 'C62xx devices, these signals may be tied directly to pins on the device, connected to on-chip peripherals, or may be disabled permanently by being tied inactive on chip. Generally, RESET and NMI are connected directly to pins on the device.

Memory/peripherals:

Synchronous external memory interface (EMIF), Two multichannel serial ports (MCSPs), Four-channel direct memory access (DMA), Two timers, X4 phase-locked-loop (PLL) option, Host-port interface (HPI), 1M-bit on-chip memory (divided into 2K by 256 bits of program memory and 64K bytes of data memory), 352-pin ball-grid array package.

Internal Memory:

The internal memory consists of 512K bits of on-chip program/cache memory and 512K bits of on-chip data memory. The program memory, configurable as cache or program, is organized in 2K of 256-bit fetch packets. The 'C6201 fetches all instructions one fetch packet at a time. The packets are processed at the maximum rate of one packet (eight 32-bit instructions) per CPU cycle or at a minimum of one instruction per cycle. The internal data memory is byte addressable by the CPU (for reads as well as writes) and supports bytes, half words, and full word transfers.

Data-Memory System

The TMS320C62xx data-memory system includes SRAM and a memory controller. The CPU can access data memory in 8-bit byte, 16-bit halfword, and 32-bit word-lengths. The data memory system supports two memory accesses per cycle. These accesses can be any combination of loads and stores from the two data buses of the CPU. Similarly, a simultaneous internal and external memory access is supported by the data memory system. The TMS320C62xx data memory system also supports direct-memory access (DMA) and external host accesses.

Program-Memory System

The TMS320C62xx program-memory system includes on-chip SRAM and a memory/cache controller. The program memory can operate as either an internal program memory or as a directly mapped program cache. There are four modes under which the program memory system operates: Program-memory mode, Cache-enable mode, Cache-freeze mode, Cache-bypass mode. The DMA can write data into an addressed space of program memory. The DMA cannot read from the internal program memory in program memory mode.

Peripherals:

In addition to on-chip memory, the TMS320C6201 contains the following peripherals:

External memory interface (EMIF)

Direct-memory access (DMA)

controller Host-port interface (HPI)

Power-down logic

Two multichannel serial ports (MCSPs)

Two 32-bit timers

External Memory Interface (EMIF)

All external data accesses by the CPU or DMA pass through the external memory interface (EMIF). The EMIF is the interface between the CPU and external memory such as synchronous dynamic random-access memory (SDRAM), synchronous-burst static RAM (SBSRAM), and asynchronous memory. The EMIF also provides 8-bit and 16-bit wide memory read capability to support low-cost boot ROM memories (flash, EEPROM, EPROM, and PROM). The interface is programmable to adapt to a variety of setup, hold, and strobe widths for asynchronous devices. SBSRAM supports zero-wait-state external access once bursts have begun. In all these types of access, the EMIF supports 8-bit, 16-bit, and 32-bit addressability for writes. All reads are performed as 32-bit transfers

Direct-Memory Access (DMA)

The on-chip DMA offers four independent, programmable channels that can be configured to transfer information from one location in the memory map to another without interfering with the operation of the CPU. This allows interfacing to slow external memories and peripherals without reducing the throughput to the CPU. The DMA controller contains its own address generators, source and destination registers, and transfer counter. The DMA has its own bus for addresses and data to keep the data transfers between memory and peripherals from conflicting with the CPU.

A DMA operation consists of a 32-bit word transfer to or from any of the three 'C62xx modules:

- Internal data memory
- Internal program memory that is not configured as cache as a destination of a transfer
- EMIF

One of the DMA channels can be used by the processor during the boot load startup procedure to initialize the internal program memory after reset. The DMA channels can be used to write to internal program memory. The boot loader uses the DMA to boot load code from off-chip memory to the internal program memory area. An external pin (sampled at reset) selects whether this boot load is performed. The serial port can also be used for booting. The DMA controller can access all internal program memory, all internal data memory, and all devices mapped to the EMIF. However, the DMA cannot use program memory as the source of a transfer, and it cannot access memories configured as cache or memory-mapped on-chip peripheral registers.

Host-Port Interface (HPI)

The HPI is a parallel port that can access the CPU memory space directly as an asynchronous interface. A host (external) processor can read from and write to the internal data memory through the 16-bit-wide access of the HPI. The HPI can boot load the CPU as well as access the full range of the 'C6201 memory. Also, the HPI offers improved performance and can operate without impacting CPU performance

Power-Down Logic

The 'C62xx supports three power-down modes (Idle1, 2, and 3) that can reduce system power requirements significantly. Idle1 halts the CPU except for the interrupt logic. Idle2 halts the CPU and the peripherals (except for the interrupt logic). Idle3 halts the phase-locked loop (PLL), stopping the clock tree from switching, which effectively halts the entire chip. Idle 3 requires a reset to wake up the device, while the other two modes can be restored using an interrupt or reset.

Multichannel Serial Port (MCSP)

The 'C6201 includes two MCSPs, supporting multivendor interface protocol (MVIP) and timers to allow easy algorithm integration. The MCSP is based on the standard TMS320C2x/C5x/C54x serial-port interface. In addition, it can buffer serial samples in memory automatically with the aid of the DMA. It also has multichannel capability compatible with the T1, E1, and MVIP standards. The MCSP provides:

- Full-duplex communication
- Double-buffered data registers
- Direct interface to other devices
- Clock generation or an internal programmable frequency shift clock
- Multichannel transmit and receive

Timers

The device has two 32-bit general purpose timers that can be used to:

- Time events
- Count events
- Generate pulses
- Interrupt the CPU
- Send synchronization events to the DMA

The timer has two signaling modes and can be clocked by an internal or an external source. The timer has an I/O pin that functions as an input clock, as an output clock, or as a general-purpose I/O pin